Excalidraw — New Developer Onboarding Guide

Everything you need to get productive on the Excalidraw codebase.

Table of Contents

1. Architecture Overview
2. Local Setup
3. Codebase Navigation
4. Key Business Flows
5. Important Components
6. Testing Guide
7. Common Pitfalls
8. Adding New Features
9. TL;DR — Day 1 Quick Start

1. Architecture Overview

Excalidraw is an open-source virtual whiteboard built with **React** and **TypeScript**. The codebase is a **Yarn workspaces monorepo** with a clear separation between the reusable library and the web application.

Tech Stack

Layer	Technology
UI Framework	React 19 + TypeScript 5.9
State Management	Jotai (atomic state) + imperative AppState
Build System	Vite (app), esbuild (packages)
Testing	Vitest + jsdom + @testing-library/react
Canvas Rendering	HTML Canvas 2D + Rough.js (hand-drawn style)
Real-time Collab	Socket.io (WebSocket)
Backend Storage	Firebase Firestore + Firebase Storage
Styling	SCSS modules
CI/CD	GitHub Actions
Containerization	Docker + nginx

Monorepo Structure

```
excalidraw/
├── excalidraw-app/          → Web application (excalidraw.com)
├── packages/
│   ├── excalidraw/          → Main React editor component (npm)
│   ├── element/             → Element types and manipulation
│   ├── common/              → Shared utilities and constants
│   ├── math/                → Geometry primitives
│   └── utils/               → Export and bounds helpers
├── examples/                → Integration examples
├── dev-docs/                → Docusaurus docs site
└── firebase-project/        → Firebase configuration
```

Package Responsibilities

Package	Role
`@excalidraw/excalidraw`	Main React component, actions, rendering, data serialization, UI
`@excalidraw/element`	Element types, Scene state, mutation, transforms, binding, groups
`@excalidraw/common`	Constants, colors, keys, event emitter, queue, utilities
`@excalidraw/math`	Points, vectors, curves, ellipses, polygons, segments
`@excalidraw/utils`	Export helpers, bounds calculations, shape utilities

Dependency Rule

Packages only depend **downward**. `common` and `math` have no internal dependencies. `element` depends on `common` + `math`. `excalidraw` depends on everything. The app depends on `excalidraw` + Firebase + Socket.io.

State Management

State is managed through two complementary systems:

- **Jotai atoms** — Fine-grained reactive state for UI. Editor uses `EditorJotaiProvider` (scoped); app adds `AppJotaiProvider`.
- **AppState** — A flat object containing all editor state (tool, theme, zoom, selection, etc.) defined in `appState.ts`.
- **Elements array** — Source of truth for drawing content. Elements are immutable; changes create new objects with incremented `version`.

Rendering Architecture

The editor uses a **multi-canvas** approach for performance:

- **StaticCanvas** — Renders committed elements (cached, only re-renders on element changes)
- **InteractiveCanvas** — Overlays selection, handles, remote cursors (re-renders on pointer move)
- **NewElementCanvas** — Renders the element being drawn (before commit)

2. Local Setup

Prerequisites

Tool	Version	Verify
Node.js	>= 18.0.0	`node --version`
Yarn	1.x (Classic)	`yarn --version`
Git	Any recent	`git --version`

Installation Steps

```
# 1. Clone
git clone https://github.com/excalidraw/excalidraw.git
cd excalidraw

# 2. Install dependencies (all workspaces)
yarn

# 3. Start dev server
yarn start

# 4. Open http://localhost:3001
```

Environment Variables

Defined in `.env.development` and `.env.production`. All prefixed with `VITE_APP_`. Defaults work for local development.

Variable	Purpose
`VITE_APP_BACKEND_V2_GET_URL`	Backend for loading shared drawings
`VITE_APP_BACKEND_V2_POST_URL`	Backend for saving shared drawings
`VITE_APP_WS_SERVER_URL`	WebSocket server for collaboration
`VITE_APP_FIREBASE_CONFIG`	Firebase config (JSON string)
`VITE_APP_PORT`	Dev server port (default: 3001)

Other Commands

Command	Purpose
`yarn build`	Build the web app
`yarn build:packages`	Build all npm packages
`yarn start:production`	Build + serve on port 5001
`yarn clean-install`	Remove node_modules + reinstall
`docker-compose up --build -d`	Run via Docker on port 3000

3. Codebase Navigation

Where to Find Things

What	Where
Element type definitions	`packages/element/src/types.ts`
Element creation	`packages/element/src/newElement.ts`
Element mutation	`packages/element/src/mutateElement.ts`
Arrow binding logic	`packages/element/src/binding.ts`
Scene state	`packages/element/src/Scene.ts`
Action system types	`packages/excalidraw/actions/types.ts`
Action manager	`packages/excalidraw/actions/manager.tsx`
All action implementations	`packages/excalidraw/actions/action*.ts(x)`
Main editor component	`packages/excalidraw/components/App.tsx`
Canvas rendering	`packages/excalidraw/renderer/staticScene.ts`
SVG export rendering	`packages/excalidraw/renderer/staticSvgScene.ts`
JSON serialization	`packages/excalidraw/data/json.ts`
Encryption	`packages/excalidraw/data/encryption.ts`
Collaboration merge	`packages/excalidraw/data/reconcile.ts`
Data migration	`packages/excalidraw/data/restore.ts`
App state defaults	`packages/excalidraw/appState.ts`
Core type definitions	`packages/excalidraw/types.ts`
Collaboration orchestrator	`excalidraw-app/collab/Collab.tsx`
WebSocket transport	`excalidraw-app/collab/Portal.tsx`
Firebase integration	`excalidraw-app/data/firebase.ts`
Local persistence	`excalidraw-app/data/LocalData.ts`
Undo/redo	`packages/excalidraw/history.ts`
Clipboard handling	`packages/excalidraw/clipboard.ts`
Library system	`packages/excalidraw/data/library.ts`

Naming Conventions

- React components: **PascalCase** (`LayerUI.tsx`)
- Utilities and modules: **camelCase** (`mutateElement.ts`)
- Actions: `action` prefix (`actionDeleteSelected.ts`)
- Jotai atoms: `Atom` suffix (`collabAPIAtom`)
- Constants: `UPPER_SNAKE_CASE`
- Type-only imports enforced: `import type { ... }`

4. Key Business Flows

Drawing an Element

```
User clicks canvas with rectangle tool
  → App.tsx: handleCanvasPointerDown()
    → newElement({type: "rectangle", x, y, ...styles})
    → NewElementCanvas renders preview

User drags
  → App.tsx: handleCanvasPointerMove()
    → Update element width/height
    → Re-render preview

User releases
  → App.tsx: handleCanvasPointerUp()
    → Commit element to Scene
    → StaticCanvas re-renders
    → History records delta
    → LocalData auto-saves
```

App Bootstrap

```
Browser loads excalidraw-app/index.tsx
  → ExcalidrawWrapper renders
  → initializeScene() runs in useEffect
    → Check URL: #room=..., #json=..., #url=..., or empty
    → Load from: collab / backend / URL / localStorage
  → Pass initialData to <Excalidraw />
  → onChange wires LocalData.save() + collab sync
```

Real-Time Collaboration

```
User clicks "Live Collaboration"
  → Collab.startCollaboration()
    → Generate roomId + encryptionKey
    → URL hash: #room=roomId,key (key never sent to server)
    → Portal.open() → Socket.io connect + join room
    → loadFromFirebase(roomId) for existing data

User draws/edits
  → onChange → syncElements()
    → reconcileElements(local, remote)
    → Portal.broadcastScene(encrypted elements)
    → queueSaveToFirebase()

Remote user's changes arrive
  → Portal receives → handleRemoteSceneUpdate()
    → reconcileElements() → updateScene()
```

Save & Export

- **Auto-save:** Every change → debounced → localStorage (appState) + IndexedDB (elements + files)
- **Save to file (Ctrl+S):** serializeAsJSON() → File System Access API or download fallback
- **Share link:** encryptData() → POST to backend → URL with #json=id,key
- **Export PNG:** renderStaticScene() on offscreen canvas → canvas.toBlob("image/png")
- **Export SVG:** renderSceneToSvg() → SVG string

Undo/Redo

- Delta-based (not full snapshots) — memory efficient
- `History` class with `undoStack` / `redoStack` of `HistoryDelta` objects
- `captureUpdate` in `ActionResult` controls whether an action is recorded
- Ctrl+Z = undo, Ctrl+Shift+Z / Ctrl+Y = redo

5. Important Components

App.tsx Critical

`packages/excalidraw/components/App.tsx`

The largest file in the project. Handles all canvas rendering orchestration, pointer/keyboard events, tool state machine, drag-and-drop, gesture handling, and collaboration state.

> Changes to App.tsx can have wide-reaching effects. Test thoroughly with different tools and input devices (mouse, touch, stylus).

ActionManager

`packages/excalidraw/actions/manager.tsx`

Registry and dispatcher for all user actions. Routes keyboard events to matching actions, executes `action.perform()`, and feeds results to the state updater.

Scene

`packages/element/src/Scene.ts`

Holds all elements and manages z-ordering via fractional indexing. Provides `getNonDeletedElements()` for rendering.

History

`packages/excalidraw/history.ts`

Delta-based undo/redo. Each `HistoryDelta` knows how to apply itself and produce an inverse.

reconcileElements()

`packages/excalidraw/data/reconcile.ts`

Merges local and remote element arrays during collaboration by comparing `version` numbers. Higher version wins; ties go to remote.

> Bugs in reconciliation cause data loss in collaborative sessions. Test concurrent editing carefully.

restore.ts

`packages/excalidraw/data/restore.ts`

Migrates saved data to current format. When the element schema changes, migration logic goes here.

Collab.tsx & Portal.tsx

`excalidraw-app/collab/Collab.tsx` / `Portal.tsx`

Manages collaboration lifecycle (start/stop, sync, file uploads) and Socket.io transport (connect, broadcast scene, broadcast cursor positions).

Firebase Integration

`excalidraw-app/data/firebase.ts`

Durable storage for collaboration (Firestore for scenes, Storage for images). Config from `VITE_APP_FIREBASE_CONFIG`.

6. Testing Guide

Commands

Command	Purpose
`yarn test:app`	Run tests in watch mode
`yarn test:update`	Run all tests + update snapshots
`yarn test:all`	Full suite: typecheck + lint + tests
`yarn test:typecheck`	TypeScript type checking
`yarn test:coverage`	Tests with coverage report

Test Infrastructure

- **Framework:** Vitest + jsdom
- **Component testing:** @testing-library/react
- **Canvas mock:** vitest-canvas-mock
- **IndexedDB mock:** fake-indexeddb
- **Setup file:** `setupTests.ts` (mocks fonts, canvas, RAF, clipboard, crypto)

Test Helpers

Helper	Location	Purpose
`render()`	`tests/test-utils.ts`	Renders Excalidraw in test container
`API`	`tests/helpers/api.ts`	Programmatic editor control (setElements, getAppState, etc.)
`UI` , `Keyboard` , `Pointer`	`tests/helpers/ui.ts`	Simulate user interactions

Writing a Test

```
import { render, unmountComponent } from "../tests/test-utils";
import { API } from "../tests/helpers/api";
import { Keyboard } from "../tests/helpers/ui";
import { Excalidraw } from "../index";

describe("my feature", () => {
  beforeEach(async () => {
    localStorage.clear();
    await render(<Excalidraw />);
  });
  afterEach(() => unmountComponent());

  it("should handle shortcut", () => {
    API.setElements([API.createElement({ type: "rectangle" })]);
    Keyboard.withModifierKeys({ ctrl: true }, () => {
      Keyboard.keyDown("a");
    });
    const state = API.getAppState();
    expect(Object.keys(state.selectedElementIds)).toHaveLength(1);
  });
});
```

Before Submitting a PR

```
yarn test:update      # Tests + snapshot updates
yarn test:typecheck   # Type checking
yarn fix              # Auto-fix lint + formatting
```

7. Common Pitfalls

1. Element Immutability

> Never mutate elements directly. Always use `mutateElement()` . Direct mutation breaks rendering, collaboration, and undo/redo.

2. Collaboration Reconciliation

Higher `version` wins; ties go to remote. Never remove deleted elements (`isDeleted: true`) from arrays — they prevent "ghost" resurrections.

3. Fractional Index Corruption

Don't manually assign `index` values. Use `Scene` methods for reordering.

4. App.tsx Side Effects

Changing pointer handlers can break drawing, selection, AND resize simultaneously. Test all tools.

5. Rendering Performance

Don't force static canvas re-render on pointer moves. Don't create new Rough.js drawables every render (they're cached). Test with 500+ elements.

6. Import Restrictions

- Inside `@excalidraw/excalidraw` : don't import from barrel `index.tsx`
- Always import Jotai from `editor-jotai.ts` , not raw `jotai`
- Cross-package imports only through `@excalidraw/*` aliases

7. Encryption Key Exposure

Keys are in the URL hash (never sent to server). Don't log full URLs in analytics/error tracking.

8. `isDeleted` vs Array Removal

Elements are soft-deleted with `isDeleted: true` . Never filter them out of the source array. Use `getNonDeletedElements()` for rendering.

9. Testing Pitfalls

- Always `await render()` — it's async
- Always `unmountComponent()` in `afterEach`
- Review snapshot diffs before committing, don't blindly update

8. Adding New Features

Adding an Action

1. Create `packages/excalidraw/actions/actionMyFeature.ts`
2. Implement the `Action` interface: `name` , `perform` , `keyTest` (optional), `PanelComponent` (optional)
3. Export from `packages/excalidraw/actions/index.ts`
4. Write tests in `actionMyFeature.test.tsx`

Adding an Element Property

1. Add to type in `packages/element/src/types.ts`
2. Set default in `packages/element/src/newElement.ts`
3. Add migration in `packages/excalidraw/data/restore.ts`
4. Wire up UI and rendering

New Feature Checklist

- ☐ Types defined in appropriate `types.ts`
- ☐ Default values set
- ☐ Migration logic added (if changing element schema)
- ☐ Action with keyboard shortcut (if applicable)
- ☐ UI wired up (toolbar, panel, menu)
- ☐ Rendering updated (if visual)
- ☐ Tests written (unit + integration)
- ☐ Snapshots updated (`yarn test:update`)
- ☐ Type checking passes (`yarn test:typecheck`)
- ☐ Linting passes (`yarn fix`)
- ☐ Works on desktop and mobile
- ☐ Works with collaboration
- ☐ Export/import handles the new data

What to Avoid

- Don't bypass the action system for undoable state changes
- Don't add dependencies to `common` or `math` on other `@excalidraw` packages
- Don't add large dependencies without discussion (bundle size is monitored)
- Don't add browser-specific APIs without fallbacks (SSR support needed)
- Don't hardcode strings — use the i18n system in `packages/excalidraw/locales/`

9. TL;DR — Day 1 Quick Start

Setup

```
git clone https://github.com/excalidraw/excalidraw.git
cd excalidraw
yarn
yarn start
# Open http://localhost:3001
```

Key Commands

Command	Purpose
`yarn start`	Dev server (port 3001)
`yarn test:update`	Tests + snapshot updates (run before committing)
`yarn test:typecheck`	TypeScript type check
`yarn fix`	Auto-fix formatting + linting
`yarn test:all`	Full CI-equivalent check

Read These Files First

1. `packages/excalidraw/index.tsx` — main entry point
2. `packages/element/src/types.ts` — the data model
3. `packages/excalidraw/types.ts` — AppState and component props
4. `packages/excalidraw/appState.ts` — default app state
5. `packages/excalidraw/actions/types.ts` — action system
6. `excalidraw-app/App.tsx` — app bootstrap

The 5 Most Important Concepts

1. **Elements are immutable** — use `mutateElement()`
2. **Actions are the state change API** — all operations go through `ActionManager`
3. **Jotai for reactive state** — atoms from `editor-jotai.ts`
4. **Multi-canvas rendering** — static (cached), interactive (selection), new element (preview)
5. **Collaboration via reconciliation** — `reconcileElements()` merges by version

Before Your First PR

```
yarn test:update      # Tests + snapshots
yarn test:typecheck   # TypeScript
yarn fix              # Formatting + linting
```